Prompting
RAG
Agents
Fine Tuning

From Basics to Production

- [x] Concepts
- [x] Diagrams
- [x] Real World Use Cases
- [x] Interview Questions
- [x] Best Practices

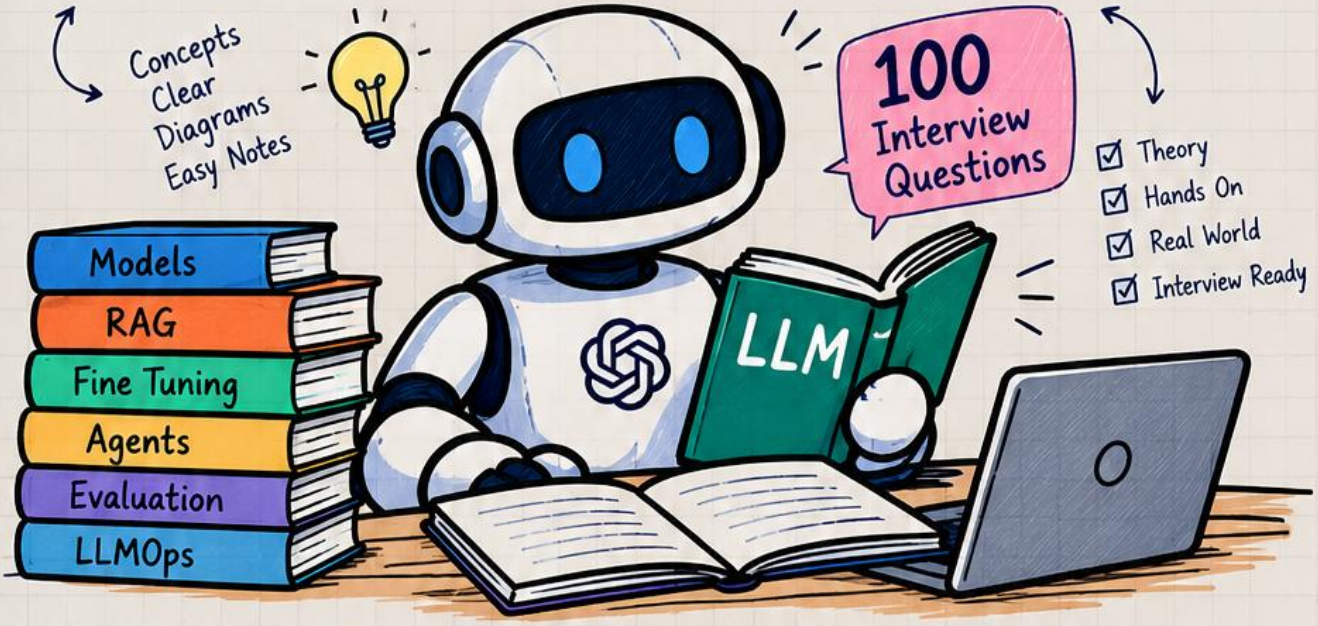
For
Students
Developers
Interview Prep

LLM HANDBOOK

Complete Notes + Cheat Sheet + 100 Interview Questions

Fundamentals | Prompting & RAG | Agents & Tool Use | Fine Tuning & Alignment | Safety & Guardrails | LLMOps & Production

Concepts
Clear
Diagrams
Easy Notes



- [x] Theory
- [x] Hands On
- [x] Real World
- [x] Interview Ready



By @codedsoul_05

LLM Handbook

1. LLM Introduction + How LLMs Work

1 What is an LLM ?

- LLM (Large Language Model) is an AI model that can understand, generate and work with human language.
- It is trained on a massive amount of text data to learn patterns, facts, grammar and reasoning.
- It can perform tasks like question answering, summarization, translation, coding, content generation and more.
- Examples : OpenAI GPT, Meta Llama, Claude, Google Gemini, Mistral.

2 Key Characteristics

- Trained on billions or trillions of tokens.
- Understands and generates natural language.
- Can perform reasoning, summarization, coding, translation, etc.
- Uses transformer architecture.
- Learns from next token prediction.
- Shows emergent abilities (can do tasks without explicit training).
- Not always factual (can hallucinate).
- Performance improves with more data and larger model size.

3 Types of LLMs (Based on Access)

- Closed Source : GPT (OpenAI), Claude (Anthropic), Gemini (Google)
- Open Source : Llama (Meta), Mistral, Falcon, etc.
- Instruction Tuned : Optimized for following instructions (e.g. ChatGPT, Llama 2 Instruct).
- Multimodal : Can work with text, images, audio, etc. (e.g. GPT-4V, Gemini).

4 What can LLMs do ?

- Answer questions.
- Summarize long documents.
- Write and debug code.
- Translate languages.
- Generate content (articles, emails, etc.).
- Solve reasoning and logic problems.
- Follow instructions.
- Work with tools (via APIs / function calling).
- Act as AI agents.

5 How do LLMs Work ? (High Level)

Input Text (Prompt) → Tokenization (split text into tokens) → Embeddings (convert tokens to vectors) → Transformer (Self-Attention + Neural Network) → Next Token Prediction (probabilities) → Output Text (response)

6 Detailed Working (Step by Step)

1. Input text is **tokenized** into smaller pieces (tokens).
2. Each token is converted into a numerical vector (embedding).
3. These vectors go into a **Transformer** model with multiple layers (self-attention + feedforward layers).
4. The model looks at the context of previous tokens using self-attention to understand the meaning.
5. It predicts the probability of the next token.
6. The predicted token is added to the sequence.
7. This process repeats until the full response is generated.

7 Next Token Prediction (Example)

Prompt : The capital of India is

Next Token	Probability
Delhi	0.65
Mumbai	0.10
India	0.05
the	0.03
...	...

Model selects the most probable token and repeats the process to generate the full response.

8 Analogy

- Think of LLM like a very advanced autocomplete.
- Instead of just a few words, it has seen billions of examples and can predict the next word in a meaningful way.
- It learns patterns from data, not hardcoded rules.

9 Real World Use Cases

- Chatbots (e.g. ChatGPT, Claude)
- Code generation (e.g. GitHub Copilot)
- Content creation (blogs, emails, marketing)
- Search and research
- Learning assistants (explain concepts)
- Automation (using APIs and tools)

LLM Handbook

2. Tokens + Tokenization + Context Window

1 What is a Token ?

- A token is a small piece of text (word, part of a word, character or symbol) that the model processes.
- LLMs do not read text like humans. They convert text into tokens and work with these tokens.
- A token can be a word (hello), part of a word (play, ##ing), or even a single character.
- Examples : hello → [hello] (1 token)
  playing → [play, ing] (2 tokens)
  unbelievable → [un, believable] (2 tokens)
  I'm → [I, ', m] (3 tokens)

2 Why Tokenization ?

- Models cannot understand raw text directly.
- Tokenization converts text into numerical tokens (token IDs).
- Helps the model handle unseen words, different languages and large vocabulary efficiently.
- Allows the model to process text of different lengths.
- Makes computation faster and memory efficient.

3 Common Tokenization Algorithms

- Byte Pair Encoding (BPE) - used by GPT, Llama, Claude.
- WordPiece - used by BERT, T5.
- SentencePiece - used by LLaMA, Mistral.
- Unigram - another probabilistic tokenization method.

4 How Does BPE Work ? (Simplified)

1. Start with characters (initial vocabulary).
2. Find the most frequent pair of characters/words.
3. Merge that pair into a new token.
4. Repeat this process for many iterations.
5. Result is a vocabulary of subword tokens.

5 Tokenization Example (BPE)

Text : "Machine learning is amazing"

Initial split : [M, a, c, h, i, n, e, , l, e, a, r, n, i, n, g, , i, s , a, m, a, z, i, n, g]

↓

After merges : [Machine , learning , is , amazing]

↓

Token IDs : [2983 , 4067 , 318 , 9821]

Actual tokenization depends on the model and tokenizer. Different models may produce different tokens.

6 Token to ID Conversion

- Each token is mapped to a unique integer ID.
- The model works with these numeric IDs.
- Example :

Token	Token ID
hello	318
world	995
!	0

These IDs are then converted to embeddings (vectors) and given to the model.

7 What is Context Window ?

- Context window is the maximum number of tokens the model can look at in a single input.
- The model can only consider tokens within this window to understand and generate the next token.
- If the input exceeds the context window, older tokens are truncated or need special techniques.
- Example : GPT-4 → 8K / 32K / 128K tokens
  Llama 3 → 8K / 32K / 70K tokens
  Claude 3 → 200K tokens

8 Example (Context Window)

- If a model has 8K context window, it can process up to 8,192 tokens at once.
- If we give 10K tokens, the first 1,808 tokens may be ignored (depending on implementation).
- Larger context windows help in long documents, code files and conversations.

9 Key Points to Remember

- Tokens are smaller pieces of text.
- Tokenization converts text into tokens using algorithms like BPE, WordPiece or SentencePiece.
- Each token has a unique ID.
- These IDs are converted to embeddings and given to the model.
- Context window limits how many tokens the model can consider at once.
- Larger context window = better for long documents, but it increases computational cost.

LLM Handbook

3. Embeddings + Vector Representations

1 What are Embeddings ?

- Embeddings are numerical vector representations of text (words, sentences or documents).
- They capture semantic meaning in a continuous vector space.
- Similar meanings are placed closer together in this space.
- Embeddings allow machines to understand the meaning and context of text.
- Generated using embedding models (e.g. OpenAI, Sentence-BERT, BGE, etc.).

2 Why Use Embeddings ?

- Convert text into numbers that machines can process.
- Capture semantic meaning (similar words → similar vectors).
- Help in search, recommendation, RAG, clustering, classification, etc.
- Work better than simple keyword matching.
- Support multilingual understanding.
- Enable efficient similarity search using vector databases.

3 How are Embeddings Generated ?

Input Text (sentence / document)
↓
Tokenization (split into tokens)
↓
Embedding Model (neural network)
↓
Dense Vector (fixed size, e.g. 768 or 1536)

- The text is tokenized.
- Tokens are passed through an embedding model (transformer based).
- The model converts the text into a dense numerical vector of fixed size.
- The vector represents the meaning of the text in a high dimensional space.

4 Example of Embeddings

Same meaning sentences have similar vectors.

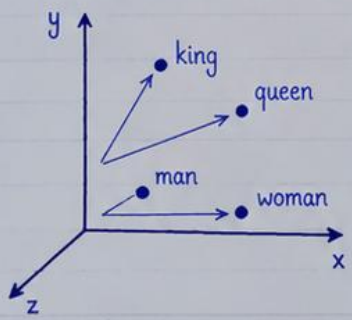
Text 1 : "I love cats" → [0.12, -0.34, 0.56, ..., 0.21]

Text 2 : "I like cats" → [0.11, -0.32, 0.55, ..., 0.20]

Text 3 : "I enjoy dogs" → [-0.40, 0.23, -0.11, ..., 0.33]

Text 1 and Text 2 have similar vectors because their meanings are similar.

5 Visual Representation (Vector Space)



Words with similar meaning are closer in the vector space.

Vector space can have hundreds or thousands of dimensions (we show 2D/3D only for easy understanding).

6 Similarity Calculation

- We use cosine similarity to measure similarity between two vectors.

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

- Value ranges from -1 to 1.
- 1 → very similar
- 0 → no similarity
- -1 → opposite meaning

7 Types of Embeddings

- Word Embeddings → single words (e.g. Word2Vec, GloVe)
- Sentence Embeddings → sentences (e.g. Sentence-BERT, OpenAI)
- Document Embeddings → longer text (e.g. BGE, E5)
- Domain Specific Embeddings → fine tuned for specific use cases (e.g. code, legal, medical)

8 Key Points to Remember

- Embeddings are dense numerical vectors.
- Similar meaning → similar vectors.
- Generated using embedding models (transformers).
- Used in search, RAG, clustering, classification, etc.
- Cosine similarity is commonly used to compare vectors.
- Vector size depends on the model (e.g. 384, 768, 1536).
- Embeddings capture meaning, not exact words.

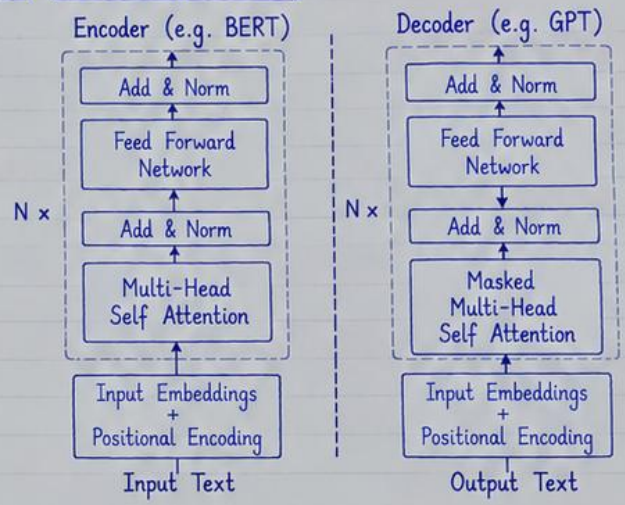
LLM Handbook

4. Transformer Architecture + Self Attention

1 What is Transformer ?

- Transformer is a deep learning architecture introduced in 2017 in the paper "Attention is All You Need".
- It is the base architecture for most modern LLMs like GPT, BERT, T5, Llama, etc.
- It uses self-attention instead of recurrence (RNN / LSTM).
- It can process all tokens in parallel, making training faster and more efficient.
- It works for both understanding (encoder) and generation (decoder) tasks.

2 Overall Architecture



3 Key Components

- Token Embeddings : convert tokens to vectors
- Positional Encoding : adds position information
- Multi-Head Self Attention : understands relationships between tokens
- Feed Forward Network : two linear layers with activation (usually GELU)
- Add & Norm : residual connection + layer normalization
- Stacked Layers : multiple transformer blocks

4 Positional Encoding

- Since transformer has no recurrence, it uses positional encoding to know the position of tokens.
- Commonly used : Sinusoidal Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos = position of token, i = dimension index
- d_{model} = embedding dimension

5 Self Attention (Intuition)

- Self attention allows each token to look at all other tokens in the input and decide which tokens are important.
- It captures long range dependencies.
- Each token creates a Query, Key and Value vector to compute attention scores with all other tokens.

6 Self Attention (Steps)

1. Convert input embeddings to Query (Q), Key (K) and Value (V) using linear layers.
2. Compute attention scores between queries and keys using dot product.
3. Scale the scores by $\sqrt{d_k}$.
4. Apply Softmax to get attention weights.
5. Multiply attention weights with values to get output.

7 Self Attention (Formulas)

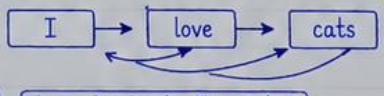
$Q = XW_Q$ (Query matrix)
$K = XW_K$ (Key matrix)
$V = XW_V$ (Value matrix)

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- X : input embeddings
- W_Q, W_K, W_V : learnable weight matrices
- d_k : dimension of key/query vectors

8 Scaled Dot Product Attention (Example)

- Suppose we have a sentence : "I love cats"
  For the token "love", attention helps it to look at "I" and "cats" to understand the context.



"love" can attend to both "I" and "cats" with different weights.

9 Key Points to Remember

- Transformer relies on self attention, not recurrence.
- Self attention captures global context efficiently.
- Multi head attention allows the model to learn multiple relationships in parallel.
- Positional encoding adds token position information.
- This architecture is used in all modern LLMs.

LLM Handbook

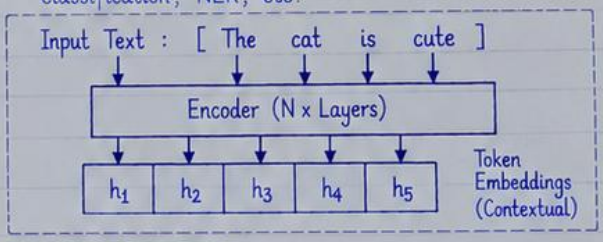
5. Encoder vs Decoder + GPT Architecture

1 Encoder vs Decoder (Overview)

- Transformer has two main types of architectures : Encoder and Decoder.
- Both are built using the same building blocks (self-attention, feed forward, etc.) but work differently.
- Decoder understands the entire input at once.
- Decoder generates the output token by token.
- Some models (like BERT) use only encoder.
- Some models (like GPT) use only decoder.
- Some models (like T5) use both (encoder-decoder).

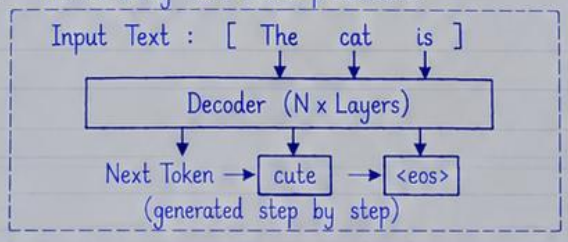
2 Encoder Architecture (e.g. BERT)

- Encoder reads the entire input sequence at once.
- Uses bidirectional self-attention (can look at both left and right tokens).
- Produces contextual embeddings for each token.
- Usually used for understanding tasks like classification, NER, etc.



3 Decoder Architecture (e.g. GPT)

- Decoder generates output one token at a time.
- Uses causal (masked) self-attention (can only look at previous tokens, not future tokens).
- Predicts the next token based on the context.
- Used for text generation, code generation, etc.
- Trained using next token prediction.

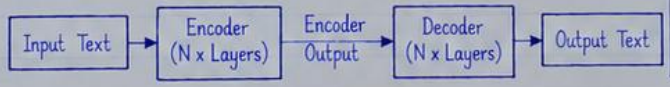


4 Key Differences

Feature	Encoder (BERT)	Decoder (GPT)
Attention Type	Bidirectional	Causal (Masked)
Input Processing	Full sequence at once	Token by token
Can see future tokens	Yes	No
Output	Contextual embeddings	Next token prediction
Common Models	BERT, RoBERTa	GPT, Llama, Mistral
Use Cases	Classification, NER, Search, Understanding	Text generation, Code generation, Chatbots

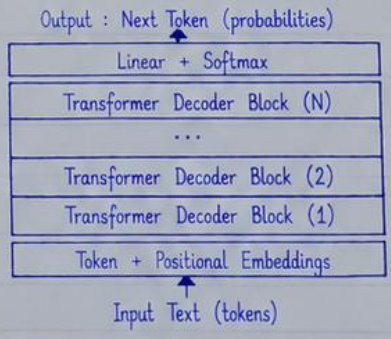
5 Encoder-Decoder Architecture (e.g. T5, BART)

- Uses both encoder and decoder.
- Encoder understands the input.
- Decoder generates the output based on encoder's output.
- Commonly used for sequence-to-sequence tasks like translation, summarization, etc.

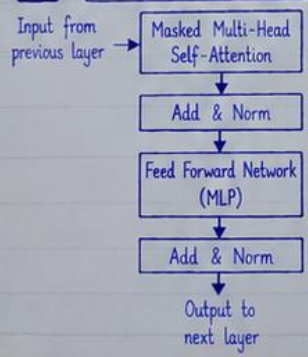


6 GPT Architecture (Decoder Only)

- GPT (Generative Pre-trained Transformer) uses only decoder blocks.
- It is a stack of multiple transformer decoder layers.
- Trained with causal self-attention and next token prediction.
- Uses large scale data and model size to learn powerful language understanding and generation capabilities.



7 GPT Decoder Block (Detailed)



- Masked Multi-Head Self-Attention : looks only at previous tokens.
- Add & Norm : residual connection + layer normalization.
- Feed Forward Network : two linear layers with activation (usually GELU).
- Repeated for many layers (e.g. 24, 32 or more).

8 Key Points to Remember

- Encoder = understand (bidirectional).
- Decoder = generate (causal).
- BERT = encoder only.
- GPT = decoder only.
- T5 = encoder-decoder.
- GPT uses causal self-attention and predicts next token.
- Transformer architecture enables parallel training.
- Scaling model size, data and compute leads to better performance.

LLM Handbook

6. Pretraining + Training Data + Scaling

1 What is Pretraining ?

- Pretraining is the first phase of training an LLM on a massive amount of unlabeled text data.
- The model learns to predict the next token given the previous tokens (self-supervised learning).
- It learns general language patterns, knowledge, reasoning and world understanding.
- No task specific labels are required.
- Example : GPT, Llama, Mistral are all pretrained on large text corpora.

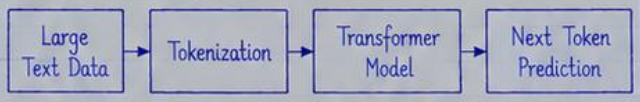
2 Pretraining Objective (Next Token Prediction)

- Given a sequence of tokens, the model predicts the probability of the next token.
- Trained using causal language modeling.
- Objective is to maximize the likelihood of the next token.
- Loss function : Cross Entropy Loss.

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log P(x_i \mid x_1, x_2, \ldots, x_{i-1})$$

N = total number of tokens
x_i = next token

3 Pretraining Pipeline

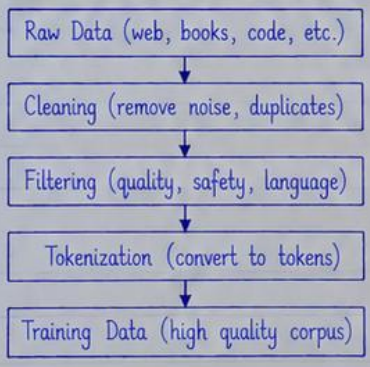


- Collect large and diverse text data.
- Clean and filter the data (remove low quality, duplicates, harmful content).
- Tokenize the text into tokens.
- Train a transformer model with next token prediction.
- Model learns useful representations and general knowledge.

4 Training Data for LLMs

- Trained on diverse and large scale internet data.
- Sources include :
  - Web pages (blogs, articles, Wikipedia)
  - Books and academic papers
  - Code from GitHub
  - Forums and discussions (Reddit, StackOverflow)
  - News, social media (filtered)
  - Other multilingual sources
- Data is cleaned, filtered and deduplicated.
- Quality and diversity of data improves model performance and reduces bias.

5 Data Preprocessing Steps



- Remove HTML, boilerplate and low quality text.
- Filter harmful, biased or spam content.
- Use language detection and deduplication.
- Tokenize using BPE or SentencePiece.
- Final dataset can contain trillions of tokens.

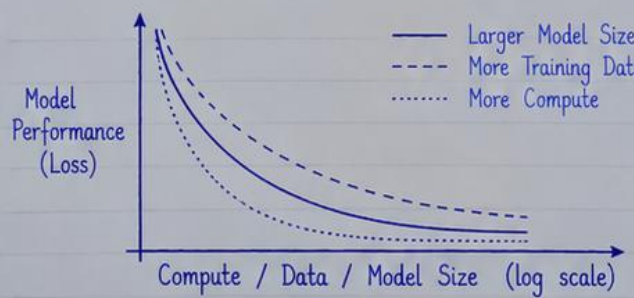
6 Scaling Laws

- Performance of LLMs improves predictably with more data, larger model size and more compute.
- Known as Scaling Laws (observed by OpenAI).

$$L(N, D, C) = AN^{-\alpha} + BD^{-\beta} + CC^{-\gamma} + \varepsilon$$

- N = model size (number of parameters)
- D = training data size (number of tokens)
- C = compute (training steps / FLOPs)
- α, β, γ = scaling exponents (empirically determined)
- ε = irreducible loss

7 Effect of Scaling (General Trend)



8 Key Takeaways

- Pretraining teaches the model general language understanding using next token prediction.
- High quality and diverse training data is crucial for better performance.
- Scaling model size, data and compute consistently improves performance.
- No task specific labels are required in pretraining.
- Pretrained models can be further improved using fine tuning, RLHF or instruction tuning.

LLM Handbook

7. Prompt Engineering + Prompting Techniques

1 What is Prompt Engineering ?

- Prompt engineering is the practice of designing and writing effective inputs (prompts) to get high quality, relevant and safe outputs from LLMs.
- It helps the model understand the task, context, format and constraints clearly.
- Good prompts can significantly improve the accuracy and usefulness of the responses.
- It is especially important for zero shot and few shot learning.

2 Why is it Important ?

- LLMs are powerful but they do not always understand vague instructions.
- A well designed prompt reduces ambiguity and guides the model to the desired output.
- Helps in getting more accurate, consistent and structured responses.
- Useful for complex tasks like reasoning, coding, summarization, data extraction, etc.
- Saves time and reduces the need for fine tuning in many cases.

3 Basic Prompt Structure

[Role / System Instruction] → Define the model's behavior

[Task / Instruction] → What you want the model to do

[Context / Input Data] → Provide necessary information

[Constraints / Format] → Rules, output format, limitations

4 Prompting Techniques

1. Zero Shot Prompting – ask directly without examples.
2. Few Shot Prompting – provide a few examples.
3. Chain of Thought (CoT) – ask the model to think step by step.
4. Role Prompting – assign a specific role (e.g. expert).
5. Instruction Prompting – clear and explicit instructions.
6. Constraints Prompting – specify what to include or avoid.
7. Format Prompting – ask for structured output (JSON, table etc).

codedsoul_05

5 Zero Shot Prompting

- Give a direct instruction without any examples.
- Works well for simple and common tasks.
- Example :

```
Explain the concept of vector database
in simple terms.
```

6 Few Shot Prompting

- Provide a few input output examples in the prompt.
- Helps the model understand the expected pattern.
- Example :

```
Input : London → Output : England
Input : Paris → Output : France
Input : Tokyo → Output : Japan
Input : Delhi → Output :
```

7 Chain of Thought Prompting (CoT)

- Ask the model to think step by step.
- Improves reasoning and complex problem solving.
- Example :

```
Solve this problem step by step :
If a train travels 120 km in 2 hours,
what is its speed ?
```

8 Role Prompting

- Assign a specific role to the model.
- Helps in getting domain specific and expert level answers.
- Example :

```
You are an expert machine learning engineer.
Explain the difference between supervised and
unsupervised learning in simple terms.
```

9 Constraints and Format Prompting

- Specify the output format, length and other constraints.
- Useful for structured data, summaries, code, etc.
- Example :

```
Summarize the following article in 5 bullet points.
Output in JSON format with keys title and point.
```

10 Key Points to Remember

- Be clear and specific in your instructions.
- Provide relevant context.
- Use examples when needed.
- Specify the desired output format.
- Set clear constraints (length, style, etc).
- Iterate and refine prompts to get better results.
- Prompt engineering is an iterative process.

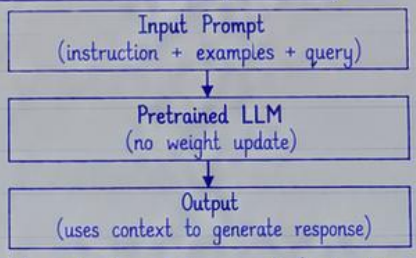
LLM Handbook

8. In Context Learning + Few Shot Prompting

1 What is In Context Learning (ICL) ?

- In Context Learning (ICL) is the ability of LLMs to learn a new task from the examples and instructions provided in the prompt without updating model weights.
- The model uses the examples and context given in the input to understand the pattern and generate the response.
- It is also called "learning from context".
- Works well because the model has already learned rich patterns during pretraining.

2 How In Context Learning Works ?



- The model reads the whole prompt (instruction, examples, and query) together.
- It finds the pattern in the examples.
- It generates the output for the new input using that pattern.
- No additional training is required.

3 Types of In Context Learning

1. Zero Shot — Only instruction, no examples.
2. Few Shot — Instruction + a few examples.
3. Many Shot — Instruction + many examples.

All are forms of In Context Learning, the difference is number of examples.

4 What is Few Shot Prompting ?

- Few Shot Prompting is a type of In Context Learning where we provide a few input output examples in the prompt.
- The model learns the pattern from these examples and applies it to the new input.
- Usually 2 to 5 examples are enough.
- Helps the model understand the task format, style and expected output.

codedsoul_05

5 Examples

Zero Shot Prompting (ICL)

```
Prompt :
Classify the sentiment of the following text.
Text: I love this product!

Output :
Positive
```

Few Shot Prompting (ICL)

```
Prompt :
Classify the sentiment of the following texts.
Text: I love this product!  → Positive
Text: This is terrible.     → Negative
Text: It is okay.           → Neutral

Text: The movie was amazing!
Output :
Positive
```

Few Shot Prompting (Another Example)

```
Prompt :
Translate English to French.
English: Hello      → French: Bonjour
English: Thank you  → French: Merci
English: Good night → French: Bonne nuit

English: See you soon
French:
```

6 Benefits of In Context Learning

- No need for model training or fine tuning.
- Flexible and quick to use.
- Works well for many tasks like translation, summarization, classification, code generation, etc.
- Can adapt to new tasks just by changing the prompt.
- Useful when labeled data is not available.

7 Limitations

- Performance depends on quality of examples.
- Too many examples can exceed context window.
- May not work well for very complex or domain specific tasks.
- Model can be sensitive to prompt format and order of examples.
- Still limited by the knowledge and capabilities learned during pretraining.
- Higher cost for longer prompts (more tokens).

8 Best Practices for Few Shot Prompting

- Use clear and specific instructions.
- Provide high quality and relevant examples.
- Keep examples consistent in format.
- Use 2 to 5 examples (more is not always better).
- Show diverse examples to cover different cases.
- Test and iterate to get better results.

9 Key Points to Remember

- In Context Learning allows learning from examples in the prompt.
- Few Shot Prompting is a type of In Context Learning.
- No model weights are updated.
- Quality and format of examples matter a lot.
- Works well for many tasks but has limitations.
- Choose the right number of examples and clear instructions.

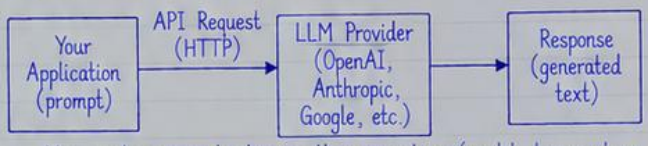
LLM Handbook

9. LLM APIs + Model Parameters + Sampling

1 What are LLM APIs ?

- LLM APIs allow us to access powerful language models (like GPT, Claude, Llama, etc.) hosted on cloud without running them locally.
- We send a request (prompt) and get a generated response from the model.
- These APIs handle model inference, scaling and infrastructure for us.
- We can use them to build chatbots, summarizers, code assistants, agents, etc.

2 How LLM APIs Work ?



- You send a prompt along with parameters (model, temperature, max tokens, etc.)
- The provider runs the model and returns the output.
- You only pay for the tokens you use (input + output).
- Popular APIs : OpenAI (GPT), Anthropic (Claude), Google (Gemini), Meta (Llama via Together / Groq), etc.

3 Example : OpenAI API (Chat Completions)

```
import openai
client = openai.OpenAI(api_key="YOUR_API_KEY")
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Explain RAG simply"}
    ],
    temperature=0.7,
    max_tokens=200
)
print(response.choices[0].message.content)
```

codedsoul_05

4 Important API Parameters

Parameter	Description
model	Which model to use (e.g, gpt-4o, claude-3, gemini)
messages / prompt	Input text with role (system, user, assistant)
temperature	Controls randomness (0 = deterministic, 1 = more random)
max_tokens	Maximum number of tokens to generate
top_p	Nucleus sampling parameter (controls diversity)
frequency_penalty	Reduces repetition of same words (range -2 to 2)
presence_penalty	Encourages new topics (range -2 to 2)
stop	Stop generation at specific tokens or sequences

5 Model Parameters (Hyperparameters)

- Model parameters are the internal weights of the model learned during training.
- They are not set by us during inference (API usage).
- Larger models have more parameters, which can learn more complex patterns.
- Examples :

Model	Parameters
GPT-3	175 Billion
Llama 2 (70B)	70 Billion
Gemma 2B	2 Billion
Phi-3	3.8 Billion

6 Sampling in LLMs

- During generation, the model gives a probability distribution over the next token. Sampling decides which token to pick.
- Different sampling methods control the tradeoff between creativity and determinism.

Example : Next Token Probabilities

Token	Probability
cat	0.4
dog	0.3
is	0.2
cute	0.1

Instead of always choosing the highest probability token, we can sample based on parameters like temperature top_p, etc.

7 Sampling Parameters

Parameter	Effect
Temperature	Controls randomness. Low (0 - 0.3) = more deterministic. High (0.7 - 1.0) = more creative.
Top-p (Nucleus)	Selects from the smallest set of tokens whose cumulative probability is ≥ p (e.g. 0.9).
Top-k	Only considers top k most probable tokens (e.g. k = 50).
Frequency Penalty	Reduces chances of repeating the same token.
Presence Penalty	Encourages the model to talk about new topics.
Max Tokens	Limits the length of the generated output.

8 Examples of Sampling Behavior (Temperature)

Temperature	Example Output (Prompt : "Write a tagline for coffee")
0.0	Coffee fuels productivity.
0.3	Coffee keeps you focused and productive.
0.7	Good ideas start with great coffee.
1.0	Coffee : because even your brain needs a break.

9 Key Takeaways

- LLM APIs provide easy access to powerful models.
- Model parameters are learned during training (not set by us).
- Sampling parameters like temperature and top_p control creativity and randomness.
- Choosing the right parameters helps get better and consistent outputs.
- Always test different settings for your use case.

LLM Handbook

10. RAG Introduction + Complete RAG Pipeline

1 What is RAG ?

- RAG (Retrieval Augmented Generation) combines information retrieval with LLM generation.
- Instead of relying only on the model's internal knowledge, it retrieves relevant information from external data sources documents, websites, databases, etc.) and uses it to generate a better and more accurate response.
- Helps reduce hallucinations and provides up to date and domain specific answers.

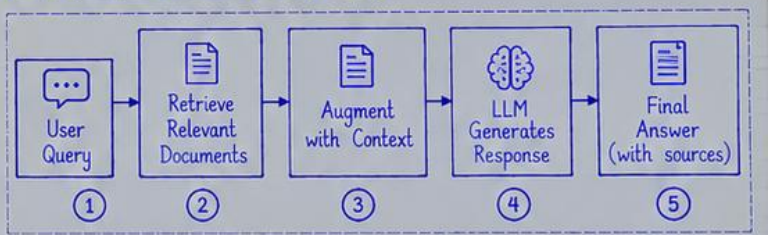
2 Why RAG is Important ?

- LLMs alone may not have the latest information.
- Useful for organization specific data (PDFs, docs, internal knowledge).
- Improves accuracy and trustworthiness.
- Can handle domain specific queries (e.g. company data, policies).
- Allows citing sources for transparency.
- No need to fine tune the model for custom data.

3 When to Use RAG ?

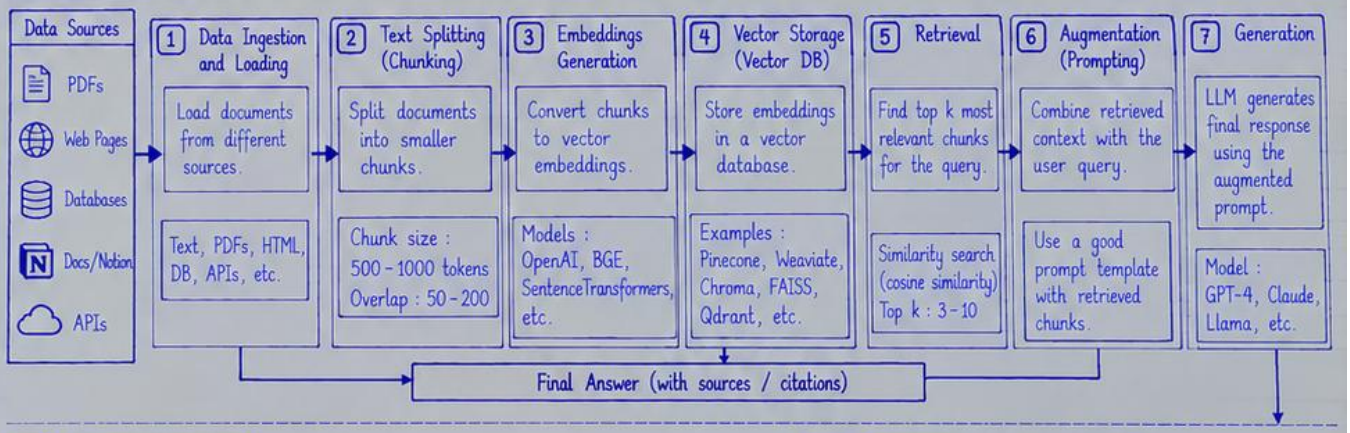
- When you have large documents or knowledge base.
- When you need up to date information.
- When answers must be based on your own data (e.g. company docs, research papers).
- When accuracy and source citation are important.
- Common use cases : chatbots, documentation search, customer support, research assistants, etc.

4 High Level RAG Pipeline

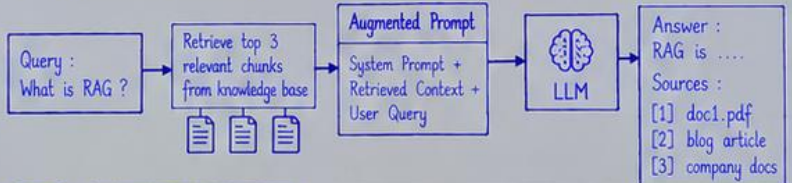


5 Complete RAG Pipeline (Detailed)

codedsoul_05



6 Example Flow



7 Tools and Technologies

Embedding Models	OpenAI, BGE, SentenceTransformers
Vector Databases	Pinecone, Weaviate, Chorma, FAISS, Qdrant
Frameworks	LangChain, LlamaIndex, Haystack
LLM Providers	OpenAI, Anthropic, Google, Meta
Data Loaders	Unstructured, LangChain loaders
Deployment	FastAPI, Streamlit, Vercel, Docker

8 Key Takeaways

- RAG combines retrieval and generation to give accurate and up to date answers.
- The pipeline involves data ingestion, chunking, embeddings, vector storage, retrieval, augmentation and generation.
- RAG reduces hallucinations and allows using your own data.
- Choice of chunk size, embedding model, vector DB and prompt template significantly affects performance.
- Always test and iterate for your use case.

LLM Handbook

11. Chunking + Embeddings + Vector Databases

1 What is Chunking ?

- Chunking is the process of splitting large documents into smaller, meaningful pieces (chunks).
- Helps LLMs handle long documents by breaking them into manageable parts.
- Each chunk is processed and embedded separately.
- Good chunking improves retrieval accuracy and reduces irrelevant results.
- Chunk size and overlap are important to maintain context and avoid losing meaning.

2 Why Chunking is Important ?

- LLMs have a limited context window.
- Large documents cannot be directly processed.
- Chunking helps retrieve only the most relevant parts.
- Improves retrieval performance and answer relevance.
- Allows efficient storage and indexing in vector databases.
- Better chunking = better embeddings = better search results.

3 Chunking Strategies

Method	Description	When to Use
Fixed Size Chunking	Split text into fixed number of tokens/characters.	Simple documents
Recursive Chunking	Split based on structure (headings, paragraphs, tokens).	General purpose (best for most cases)
Semantic Chunking	Split based on meaning (using embeddings or NLP).	Complex documents
Sliding Window	Overlapping chunks to maintain context.	When context continuity is important

4 Chunking Parameters

Parameter	Description
chunk_size	Number of tokens or characters in each chunk (e.g. 500 - 1000 tokens).
chunk_overlap	Overlap between consecutive chunks to preserve context (e.g. 50 - 200 tokens).
separators	Characters or patterns to split text (e.g. \n\n, headings, sentences).
strategy	Fixed, recursive, semantic, etc.
metadata	Store extra information (source, page number, section, etc.).

codedsoul_05

5 What are Embeddings ?

- Embeddings are numerical vector representations of text, images or other data.
- They capture the semantic meaning of the content.
- Similar content has similar vectors (closer in vector space).
- Embeddings are generated using pre trained models (e.g. OpenAI, SentenceTransformers).
- Embeddings convert text into high dimensional vectors (e.g. 768, 1024, 1536 dimensions).

6 Embedding Models (Popular)

Model	Dimensions	Use Case
OpenAI text-embedding-3-small	1536	General purpose, cost effective
OpenAI text-embedding-3-large	3072	Higher accuracy, complex tasks
sentence-transformers/all-MiniLM-L6-v2	384	Open source, lightweight
BAAI/bge-base-en	768	Good performance, open source
BAAI/bge-large-en	1024	Higher quality embeddings

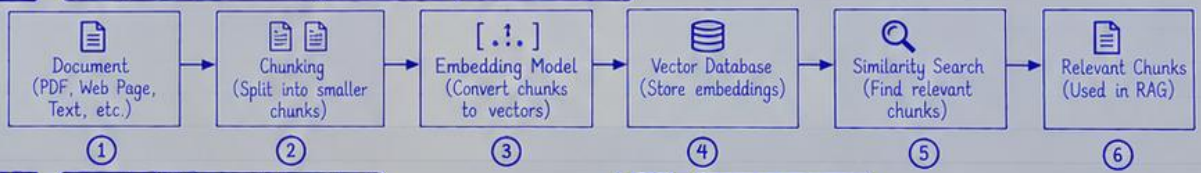
7 What are Vector Databases ?

- Vector databases store and search embeddings efficiently.
- They allow similarity search to find the most relevant chunks for a query.
- Designed to handle high dimensional vectors.
- Commonly used in RAG systems, semantic search and recommendation systems.

8 Popular Vector Databases

Database	Type	Key Features	Use Case
Pinecone	Cloud	Easy to use, scalable	Production RAG apps
Weaviate	Open Source	Hybrid search, metadata	RAG, semantic search
Chroma	Open Source	Lightweight, simple	Small to medium projects
FAISS	Library	Fast similarity search	Research, local use
Qdrant	Open Source	High performance	Production, large scale
Milvus	Open Source	Scalable, multi modal	Enterprise use

9 Complete Flow (Chunking + Embeddings + Vector DB)



10 Example (Python with LangChain)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

documents = load_documents("docs/")
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(documents)
embeddings = OpenAIEmbeddings()
vector_db = Chroma.from_documents(chunks, embeddings)
```

11 Key Takeaways

- Chunking breaks large documents into smaller, meaningful pieces.
- Good chunk size and overlap improve retrieval quality.
- Embeddings convert text into numerical vectors that capture meaning.
- Vector databases store embeddings and enable fast similarity search.
- Choosing the right chunking strategy, embedding model and vector database is important for a high quality RAG system.
- Always experiment and evaluate different settings for your use case.

LLM Handbook

12. Retrieval + Reranking + Hybrid Search

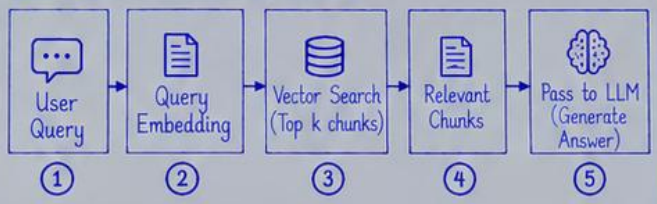
1 What is Retrieval ?

- Retrieval is the process of finding the most relevant information (chunks) from a large collection of documents for a user query.
- It uses embeddings and vector similarity to search for relevant chunks.
- The retrieved chunks are then passed to the LLM as context to generate a better and more accurate response.
- Good retrieval is the foundation of a successful RAG system.

2 Why is Retrieval Important ?

- LLMs do not have access to your private or latest data.
- Retrieval helps bring relevant and up to date information from external sources.
- Improves accuracy, reduces hallucinations and provides source grounded answers.
- Useful for domain specific applications (company data, research papers, internal docs, etc).
- Retrieval quality directly impacts the final answer quality.

3 Basic Retrieval Flow



4 Types of Retrieval

Type	Description
Dense Retrieval	Uses embeddings and vector similarity (e.g. cosine similarity).
Sparse Retrieval	Uses keyword based matching (e.g. BM25).
Hybrid Retrieval	Combines dense and sparse retrieval for better results.

codedsoul_05

5 Reranking

- Reranking is the process of reordering the retrieved chunks to keep the most relevant ones at the top.
- Initial retrieval may return many chunks, some of which are not highly relevant.
- A reranker model (cross encoder) scores each chunk with the query and sorts them based on relevance.
- Helps improve answer quality by removing noisy or irrelevant chunks.

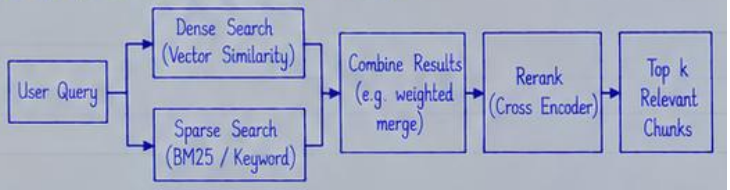
6 Retrieval vs Reranking

Retrieval (Initial Search)	Reranking (Reorder Results)
Finds relevant chunks from database	Reorders retrieved chunks by relevance
Uses fast methods (dense, sparse or hybrid)	Uses more accurate but slower models (e.g. cross encoder)
Returns top k chunks (e.g. 20-50)	Selects best k chunks (e.g. top 3-5)
Focuses on speed and coverage	Focuses on accuracy and relevance

7 Hybrid Search

- Hybrid search combines dense retrieval (embeddings) with sparse retrieval (keyword search like BM25).
- Dense search understands semantic meaning.
- Sparse search works well for exact keyword matches and rare terms.
- Hybrid search gives better and more robust results by combining both methods.

8 Hybrid Search Architecture



9 Popular Tools and Libraries

Tool / Library	Use Case
FAISS	Vector search (fast, in memory)
Pinecone	Managed vector database (cloud)
Weaviate	Vector DB with hybrid search support
Qdrant	Open source vector database (hybrid search)
Elasticsearch	Sparse search (BM25) and hybrid search
LanceDB	Lightweight vector DB for local use
Voyage AI	Reranking models (high quality)
Cohere Rerank	Reranking API (easy to use)
LangChain / LlamaIndex	Frameworks to build retrieval and reranking pipelines

10 Key Takeaways

- Retrieval helps find relevant information for the query.
- Reranking improves the quality of retrieved results by reordering them.
- Hybrid search combines dense and sparse retrieval for better accuracy.
- Use a good reranker (e.g. cross encoder) for high quality results.
- Experiment with different top k values (initial retrieval and final selection).
- Choose the right vector database and search method based on your use case.
- Good retrieval and reranking significantly improve the performance of RAG systems.

LLM Handbook

13. Fine Tuning + SFT + PEFT + LoRA

1 What is Fine Tuning ?

- Fine tuning is the process of further training a pretrained LLM on a smaller, task specific dataset to adapt it to a particular domain or use case.
- The model already has general knowledge from pretraining, we just update it slightly for our needs.
- Helps the model follow instructions better, understand domain specific language and generate more relevant outputs.
- Examples: customer support bot, medical assistant, legal document analysis, coding assistant.

2 Why Fine Tuning is Important ?

- Pretrained models are general purpose and may not perform well on your specific task.
- Fine tuning improves accuracy and relevance for domain specific use cases.
- Helps the model learn your data, style, tone and format.
- Can reduce the need for long prompts.
- Useful when you have labeled data and consistent tasks.
- Can be more cost effective for high volume use cases (compared to long prompts every time).

3 Types of Fine Tuning

Type	Description
Full Fine Tuning	Update all model parameters. High compute and memory required.
SFT (Supervised Fine Tuning)	Train the model on labeled instruction input output pairs.
PEFT (Parameter Efficient Fine Tuning)	Update only a small portion of parameters using techniques like LoRA, adapters, prefix tuning, etc.
LoRA (Low Rank Adaptation)	A popular PEFT method which adds small trainable matrices to existing weights.

codedsoul_05

4 Supervised Fine Tuning (SFT)

- SFT trains the model using labeled instruction and response pairs.
- Dataset consists of input (instruction) and expected output (response).
- Helps the model learn to follow instructions, adhere to a specific format and behave consistently.
- Commonly used for chatbots, domain specific assistants and task specific models.

Example SFT Data Format

Input (Instruction)	Output (Response)
Summarize the following text in 2 lines Text: Large Language Models are powerful AI systems ...	Large Language Models are powerful AI systems that can understand and generate human like text.

5 Parameter Efficient Fine Tuning (PEFT)

- PEFT methods update only a small number of parameters instead of the entire model.
- Reduce computational cost, memory usage and training time.
- Maintain the performance close to full fine tuning.
- Common PEFT techniques: LoRA, Adapters, Prefix Tuning, Prompt Tuning, QLoRA, etc.
- Useful for large models where full fine tuning is not feasible.

6 LoRA (Low Rank Adaptation)

- LoRA adds small trainable low rank matrices to the existing model weights.
- Keeps the original model weights frozen.
- Significantly reduces the number of trainable parameters.
- Helps achieve good performance with low compute and memory.
- Widely used for fine tuning large models like Llama, Mistral, Gemma, etc.

7 Full Fine Tuning vs PEFT vs LoRA

Aspect	Full Fine Tuning	PEFT (General)	LoRA
Parameters Updated	All parameters	Small subset	Only low rank matrices
Compute Cost	Very high	Low	Low
Memory Usage	Very high	Low	Low
Training Time	Long	Short	Short
Performance	Best (but costly)	Good	Good to Very Good
Use Case	When resources are available	Most real world use cases	Popular choice for LLM fine tuning

8 Simple LoRA Flow



9 Popular Tools and Libraries

Tool / Library	Use Case
Hugging Face Transformers	Model loading and training
PEFT (Hugging Face)	LoRA, adapters, prefix tuning, etc.
TRL	SFT and RLHF training utilites
LlamaFactory	Easy fine tuning for LLMs
Unsloth	Fast and memory efficient fine tuning (LoRA, QLoRA)
BitsAndBytes	4 bit and 8 bit training (QLoRA)

10 Key Takeaways

- Fine tuning adapts a pretrained model to your specific task.
- SFT uses labeled instruction and response pairs.
- PEFT methods update only a small number of parameters.
- LoRA is a popular and efficient PEFT technique.
- Fine tuning improves accuracy, relevance and consistency.
- Choose the right method based on your data size, resources and use case.
- Always experiment and evaluate the model on real examples to get the best results.

LLM Handbook

14. RLHF + DPO + Alignment

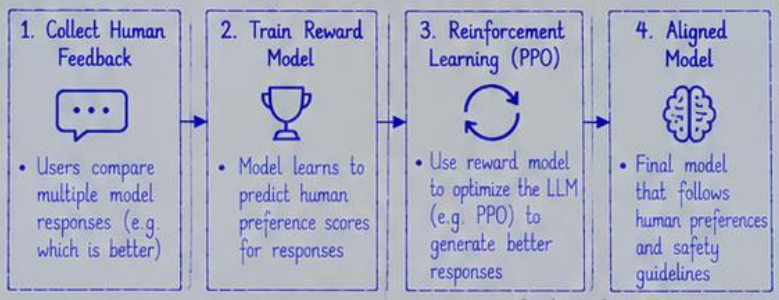
1 What is RLHF ?

- RLHF (Reinforcement Learning from Human Feedback) is a method to align LLMs with human preferences.
- We collect human feedback on model outputs and use reinforcement learning to train the model to generate more helpful, truthful and safe responses.
- Helps reduce harmful, biased and irrelevant outputs.
- Widely used in models like ChatGPT, Claude, Gemini, etc.

2 Why Alignment is Important ?

- Pretrained models can generate incorrect, harmful or biased outputs.
- Alignment ensures the model behaves according to human values, safety guidelines and intended use cases.
- Reduces toxic, misleading and hallucinated content.
- Builds trust and reliability for real world applications.
- Helps follow ethical, legal and domain specific rules.

3 RLHF Training Pipeline (Overview)



4 Reward Model

- A separate model that predicts how good a response is based on human feedback.
- Trained on pairs of responses (preferred vs non preferred).
- Outputs a reward score for a given prompt and response.
- Used during RL to guide the model towards better outputs.

codedsoul_05

5 Reinforcement Learning with Human Feedback (RLHF)

- Uses the reward model to fine tune the LLM with reinforcement learning (commonly PPO - Proximal Policy Optimization).
- Optimizes the model to maximize the reward score.
- Helps generate responses that are more helpful, harmless and consistent with human preferences.
- Can be computationally expensive and involve multiple training steps.

6 Direct Preference Optimization (DPO)

- DPO is a simpler and more efficient alternative to RLHF.
- Instead of using reinforcement learning, DPO directly optimizes the model using preference data.
- Trains the model to prefer the chosen response over the rejected one using a pairwise loss.
- Easier to train, more stable and often gives comparable results to RLHF.

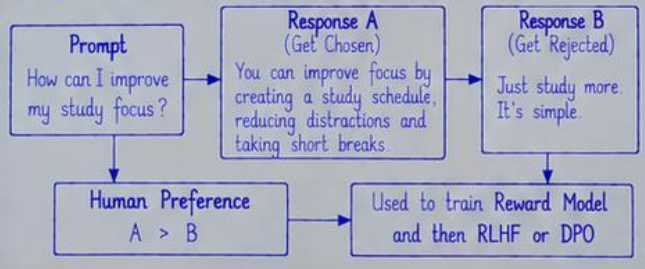
7 RLHF vs DPO (Comparison)

Aspect	RLHF (PPO)	DPO
Method	Reinforcement Learning	Direct Optimization
Training Steps	Multiple (Reward Model + RL)	Single stage
Complexity	High	Lower
Compute Cost	High	Lower
Stability	Can be unstable	More stable
Performance	Very good	Comparable
Use Case	Large scale models	Efficient fine tuning

8 Alignment Techniques

Technique	Description
RLHF	Uses human feedback + reinforcement learning to optimize the model.
DPO	Directly optimizes model with preference pairs.
Constitutional AI	Uses a set of principles or rules to guide outputs.
Safety Tuning	Fine tunes on safety oriented datasets.
Red Teaming	Tests the model with adversarial prompts to find failures.
Guardrails	Adds filtering and monitoring at inference time.

9 Alignment Example (Preference Data)



10 Key Takeaways

- RLHF aligns LLMs using human feedback and reinforcement learning.
- DPO is a simpler and efficient alternative that directly optimizes preference data.
- Alignment is essential for safe, helpful and trustworthy AI systems.
- Reward models, safety tuning and red teaming improve model behavior.
- Different techniques (RLHF, DPO, constitutional AI, guardrails) can be combined for better results.
- Always test your model with real world prompts and edge cases.
- Alignment is an ongoing process, not a one time step.

LLM Handbook

15. Function Calling + Tool Use

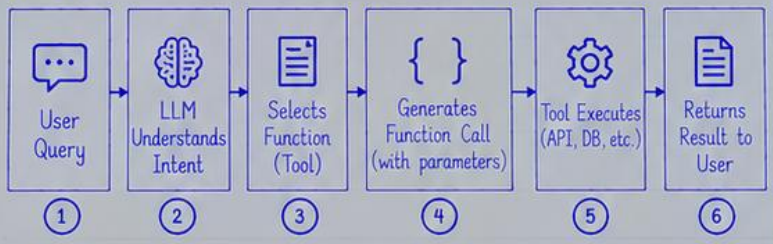
1 What is Function Calling ?

- Function calling allows the LLM to call external functions or tools to perform real world actions.
- The model understands the user intent, selects the right function, fills the required parameters and returns a structured function call.
- Instead of only generating text, the LLM can interact with real systems like APIs, databases, calculators, search engines, etc.
- Helps build more accurate, reliable and actionable AI applications.

2 Why is Function Calling Important ?

- Extends the capabilities of LLMs beyond text generation.
- Allows access to real time data (weather, stock prices, etc.).
- Helps perform actions (send email, create a booking, update database, etc.).
- Reduces hallucinations by using trusted external tools.
- Enables building AI agents and production ready applications.
- Useful for domain specific use cases like finance, healthcare, travel, customer support, etc.

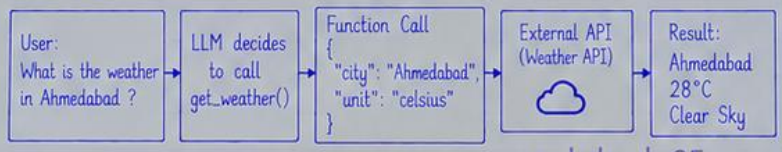
3 How Function Calling Works ?



4 Example Function Definition (OpenAI)

```
functions = [
    {
      "name": "get_weather",
      "description": "Get current weather
      for a given city",
      "parameters": {
        "type": "object",
        properties": {
          "city": {"type": "string", "description": "City name"},
          "unit": {"type": "string", "enum": ["celsius", "fahrenheit"],
            "description": "Temperature unit"}
        },
        "required": ["city"]
      }
]   }
```

5 Example Flow (Weather Forecast)



codedsoul_05

6 Tool Use vs Normal LLM Response

Aspect	Normal LLM Response	Function Calling + Tool Use
Information Source	Only training data	Real time external data
Accuracy	Can be outdated	More accurate and reliable
Capabilities	Text generation only	Can perform real world actions
Use Cases	General knowledge QnA	Real time data, automation, domain specific tasks
Hallucination Risk	Higher	Lower (uses trusted tools)

7 Common Tools and Use Cases

Tool Type	Examples	Use Case
Web APIs	Weather API, News API	Real time information
Search	Google Search, Tavily	Fetch latest web data
Databases	PostgreSQL, MongoDB	Read and write data
Productivity	Gmail, Calendar, Notion	Send email, create events
Code Execution	Python REPL	Run code, calculations
Custom Tools	Your own functions	Domain specific actions

8 Libraries and Frameworks

Library / Framework	Purpose	Use Case
OpenAI Function Calling	Native function calling support	ChatGPT, GPT-4, GPT-4o
LangChain	Tool use and agent framework	Build AI agents
LlamaIndex	Connect tools and data sources	RAG + tool use
AutoGen	Multi agent tool use	Complex workflows
CrewAI	Agent orchestration	Multi agent systems
Haystack	Tools and connectors	Enterprise use cases

9 Example (Python with OpenAI)

```
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
  model="gpt-4o",
  messages=[{"role": "user", "content": "What is the weather
  in Ahmedabad today?"}],
  functions=functions,
  function.call="auto"
)
print(response.choices[0].message)
```

10 Key Takeaways

- Function calling allows LLMs to interact with external tools and perform real world actions.
- Helps reduce hallucinations and improves reliability.
- Useful for real time data, automation and domain specific applications.
- Widely supported in models like GPT-4, Claude and open source frameworks.
- Essential concept for building AI agents and production ready LLM applications.
- Choose the right tools and design clear function schemas for better results.

LLM Handbook

16. AI Agents + Agentic Workflows

1 What are AI Agents ?

- AI Agents are systems that can perceive, plan, reason and take actions to achieve a goal.
- Unlike simple LLM responses, agents can use tools, access external data, maintain memory and work over multiple steps.
- They can autonomously decide what to do next based on the task and environment.
- Examples: research assistant, travel planner, coding agent, customer support agent.

2 Why are AI Agents Important ?

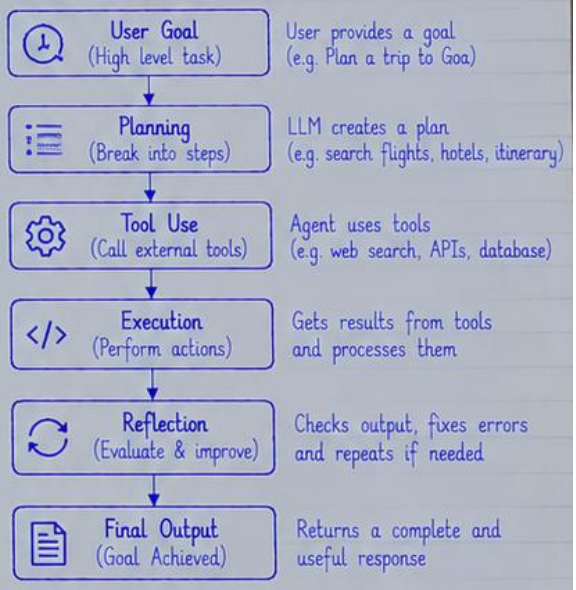
- Can handle complex, multi step tasks.
- Increases productivity and automation.
- Can interact with real world tools and data sources.
- Useful for personal assistants, business workflows and domain specific applications.
- Can plan, execute, evaluate and refine results.
- Foundation for next generation AI systems (autonomous workflows, AI employees, etc.).

3 Key Components of an AI Agent

Component	Description
LLM (Brain)	Understands the task, reasons and decides next step
Planning	Breaks the goal into smaller steps (plan and replan)
Memory	Stores conversation history, context and past actions
Tools	Uses external tools (APIs, search, databases, code, etc.).
Execution	Performs actions using tools and gets results
Observation	Analyzes results and decides next step
Feedback Loop	Iterates until goal six achieved

codedsoul_05

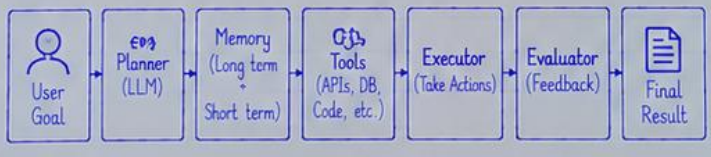
4 How an AI Agent Works ? (Flow)



5 Types of AI Agents

Type	Description	Example
Simple Agent	Performs single step actions	Chatbot with tool use
Reactive Agent	Responds based on current input	Auto email responder
Planning Agent	Breaks task into steps and plans	Research assistant
Multi Agent	Multiple agents working together	Team of agents for complex tasks
Autonomous Agent	Works with minimal human input	AI employee / full workflow agent

6 Agentic Workflow Architecture



7 Popular Frameworks and Libraries

Framework / Library	Use Case
LangChain	Build agents and tool use workflows
LangGraph	Create complex agent workflows (stateful)
CrewAI	Multi agent systems
AutoGen	Multi agent conversations (Microsoft)
LlamaIndex	Agents with data retrieval (RAG + tools)
Vertex AI Agent Builder	Build production ready agents (Google)
Haystack Agents	Open source agentic pipelines

8 Example Agentic Workflow (Travel Planner)



9 Real World Use Cases

- Travel planning assistant
- Research and summarization agent
- Code generation and debugging agent
- Customer support agent
- Personal productivity assistant
- Business automation workflows
- Multi agent systems for complex tasks
- Autonomous AI employees

10 Key Takeaways

- AI Agents go beyond simple text generation and can plan, use tools and take actions.
- They combine LLMs, memory, tools and reasoning to complete real world tasks.
- Agentic workflows can be simple or complex (single agent or multi agent).
- Frameworks like LangChain, LangGraph, CrewAI and AutoGen make it easier to build agents.
- Always define clear goals, choose the right tools and evaluate results for better performance.
- AI agents are a rapidly growing field and will power the next generation of AI applications.

LLM Handbook

17. LLM Evaluation + LLM as a Judge

1 What is LLM Evaluation ?

- LLM evaluation is the process of measuring the quality, correctness, safety and usefulness of LLM outputs.
- It helps us understand how well the model performs on different tasks and real world use cases.
- Evaluation is important because LLMs can produce incorrect, biased or hallucinated outputs.
- Good evaluation ensures reliable, safe and helpful AI applications.

2 Why is LLM Evaluation Important ?

- LLM outputs can be unpredictable and inconsistent.
- Helps compare different models and select the best one.
- Measures accuracy, relevance, helpfulness and safety.
- Reduces hallucinations and harmful content.
- Critical for production systems and real world applications.
- Useful for tracking model improvements over time.

3 Key Evaluation Metrics

Metric	Description
Accuracy	Correctness of the answer
Relevance	How relevant the answer is to the query
Faithfulness	Whether the answer is grounded in the source
Helpfulness	Useful and complete response
Coherence	Logical and well structured answer
Conciseness	Not too long and not too short
Safety / Harmlessness	Does not contain harmful or biased content
Hallucination Rate	Measures incorrect or fabricated information
Latency	Time taken to generate response
Cost	Token usage and API cost

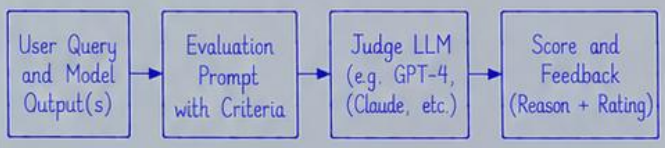
4 Types of Evaluation

Type	Description
Automatic Evaluation	Uses metrics or LLM as a judge
Human Evaluation	Real human annotators review outputs
Rule Based Evaluation	Predefined rules and patterns
Dataset Based Evaluation	Uses standard benchmark datasets
Task Specific Evaluation	Custom metrics for specific use cases (e.g. RAG, chat, code, etc.)
Pairwise Comparison	Compare two model responses and choose the better one

5 LLM as a Judge

- LLM as a judge uses another LLM to evaluate and score model outputs.
- The judge model compares responses based on criteria like correctness, relevance, helpfulness, safety, etc.
- It is cost effective and scalable compared to human evaluation.
- Widely used for RAG evaluation, chatbot evaluation and model comparison.

6 How LLM as a Judge Works ?



7 Example Judge Prompt

```
You are an expert evaluator.
Evaluate the following AI response based on the criteria.

Criteria:
- Correctness (0-5)
- Relevance (0-5)
- Helpfulness (0-5)
- Safety (0-5)

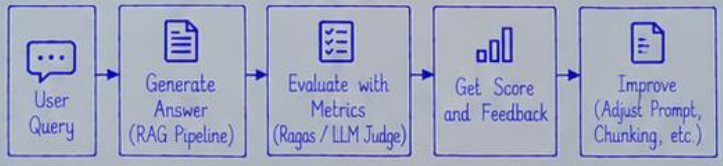
User Query: <query>
Model Response: <response>
Give a score for each criteria with a short reason and final verdict.
```

codedsoul_05

8 Popular Evaluation Frameworks and Tools

Tool / Library	Use Case
OpenAI Evals	Evaluate LLMs with predefined datasets
Ragas	RAG specific evaluation (answer relevancy, faithfulness, context usage)
TruLens	Evaluate and debug LLM applications
LangSmith	Logging, evaluation and monitoring for LLM apps
DeepEval	Evaluate LLM outputs with custom metrics
Helm (Stanford)	Benchmark LLMs on multiple scenarios
LlamaIndex Evaluators	RAG evaluation with built in metrics
Anthropic Constitutional AI	Safety and alignment evaluation

9 Example Evaluation Flow (RAG)



10 Key Takeaways

- LLM evaluation is essential for building reliable AI systems.
- Use a combination of automatic metrics and human evaluation.
- LLM as a judge provides a scalable and cost effective way to evaluate outputs.
- Choose the right metrics and tools based on your use case.
- Always analyze failure cases and continuously improve.
- Evaluation is an ongoing process, not a one time step.

LLM Handbook

18. Hallucination + Guardrails + Safety

1 What is Hallucination ?

- Hallucination occurs when an LLM generates information that is incorrect, misleading or not supported by real facts or context.
- The model produces confident but false answers because it predicts the next token, it does not "know" truth.
- Can happen due to lack of knowledge, ambiguous prompts, outdated training data or trying to be helpful.
- Types: Factual, Contextual, Citation and Logical Hallucination.

2 Why is it a Problem ?

- Leads to incorrect and misleading information.
- Can cause real world harm in healthcare, legal, finance and other critical domains.
- Undermines trust and reliability of AI systems.
- Can spread misinformation and bias.
- Especially risky in production systems and automated agents.
- Needs proper detection and mitigation strategies.

3 Types of Hallucinations

Type	Description	Example
Factual Hallucination	Incorrect facts or made up information.	"The first man to visit Mars was Elon Musk in 2015."
Contextual Hallucination	Answer not relevant to the given context.	User asks about React, model starts explaining Python.
Citation Hallucination	Invented references or fake citations.	"According to a 2023 Stanford paper..." (paper does not exist)
Logical Hallucination	Answer that sounds correct but is logically wrong.	"A triangle can have four sides if one side is very small."

codedsoul_05

4 Common Causes

- Training data contains outdated or incorrect information.
- Model trise to be helpful and fills in missing details.
- Ambiguous or unclear user prompts.
- Lack of grounding with real time or trusted knowledge sources.
- Over generalization from patterns in training data.
- Creating content on unseen or domain specific topics.

5 Impact of Hallucination

- Misleads users and causes wrong decisions.
- Legal and compliance risks.
- Harmful in sensitive domains (healthcare, finance, law).
- Damages brand trust.
- Can propagate misinformation at scale.

6 What are Guardrails ?

- Guardrails are techniques and policies used to control and constrain LLM behavior.
- They help ensure the model generates safe, relevant, accurate and ethical responses.
- Guardrails can be implemented at the prompt level, application level or model level.
- They act as a safety layer between user input and model output.

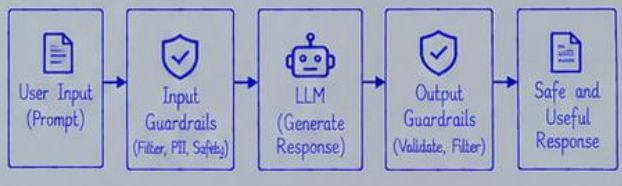
7 Types of Guardrails

Type	Description	Examples
Input Guardrails	Filter or sanitize user input.	Block harmful prompts, remove PII, detect intent
Output Guardrails	Check and filter model output.	Toxicity filter, fact check, format enforcement
Policy Guardrails	Enforce rules and guidelines.	Domain specific rules, compliance policies
System Guardrails	Architecture level controls.	RAG with trusted sources, human in the loop, monitoring

8 Safety Techniques

Technique	Description	Use Case
Content Filtering	Detect and block harmful content.	Toxic, abusive or illegal content
Safety Classifiers	ML models to classify risk.	Hate speech, violence, PII
Prompt Engineering	Use clear instructions and constraints.	Limit scope and behavior
RAG (Grounding)	Use trusted external sources.	Reduce factual hallucination
Human in the Loop	Human review for critical outputs.	High risk decisions
Red Teaming	Test with adversarial prompts.	Find vulnerabilities
Monitoring	Track model behavior in production.	Detect and fix issues early

9 Example Guardrail Flow



10 Safety Use Cases

- Preventing generation of harmful, toxic or biased content.
- Blocking sensitive information like personal data (PII).
- Ensuring factual accuracy using RAG and trusted sources.
- Following domain specific compliance rules (healthcare, finance, legal).
- Preventing prompt injection attacks.
- Monitoring and logging model interactions.
- Providing safe AI assistants for enterprise and public use.

11 Key Takeaways

- Hallucination is a major challenge in LLMs and can lead to incorrect or harmful outputs.
- Guardrails are essential to ensure safe, reliable and trustworthy AI systems.
- Use a combination of techniques: input and output filters, RAG, safety classifiers, prompt engineering and human review.
- Always test with adversarial prompts and monitor in production.
- Safety is an ongoing process, not a one time step.
- Choose the right guardrails based on your use case and domain.

LLM Handbook

19. LLMOps + Monitoring + Cost + Production

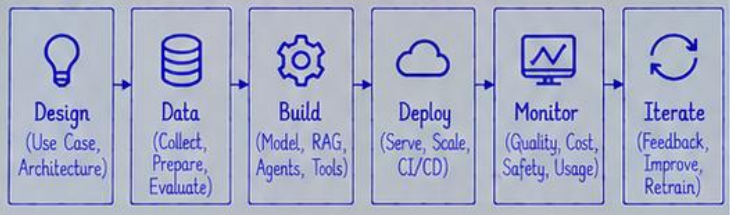
1 What is LLMOps ?

- LLMOps (Large Language Model Operations) is the set of practices, tools and processes to build, deploy, monitor and maintain LLM applications in production.
- It covers the full lifecycle from development to monitoring, evaluation, scaling and cost optimization.
- Helps make LLM applications reliable, scalable, safe and cost efficient.

2 Why is LLMOps Important ?

- LLM applications are complex and can be unpredictable.
- Need continuous monitoring for quality, safety and cost.
- Helps detect issues like hallucinations, prompt failures, toxic outputs or performance degradation.
- Enables faster iteration and experimentation.
- Improves reliability, user trust and production readiness.
- Essential for real world use cases with millions of users.

3 LLMOps Lifecycle (End to Ed)



4 Key Components of LLMOps

Component	Description
Data Management	Manage and evaluate training and retrieval data.
Experimentation	Track experiments, prompts and model versions.
Deployment	Serve models using APIs or platforms.
Monitoring	Track performance, quality, safety and cost.
Evaluation	Use metrics and LLM as a judge.
Feedback Loop	Collect user feedback and improve.
Security & Compliance	Handle data privacy and regulatory requirements.
Cost Optimization	Reduce token usage and infrastructure cost.

5 Deployment Options

Option	Description	Use Case
OpenAI API	Managed API (GPT-4, GPT-4o)	Quick deployment, scalable
Self Hosted	Run open source models (Llama)	More control, lower cost (long term)
Cloud Platforms	AWS, Azure, GCP (SageMaker, Bedrock, Vertex AI)	Enterprise applications
Inference Tools	vLLM, TGI, Ollama	High performance inference
Serverless	Vercel, Cloud Run	Small to medium scale apps

6 Monitoring in LLMOps

Aspect	What to Monitor ?
Quality	Relevance, correctness, helpfulness (LLM as a judge)
Safety	Toxicity, hallucinations, harmful content
Performance	Latency, throughput, error rate
Cost	Token usage, API cost, infrastructure cost
User Behavior	Usage patterns, feedback, drop offs
System Health	Uptime, API failures, queue length, GPU usage
Data Drift	Changes in input data distribution
Prompt Drift	Changes in prompt effectiveness over time

7 Popular Monitoring Tools

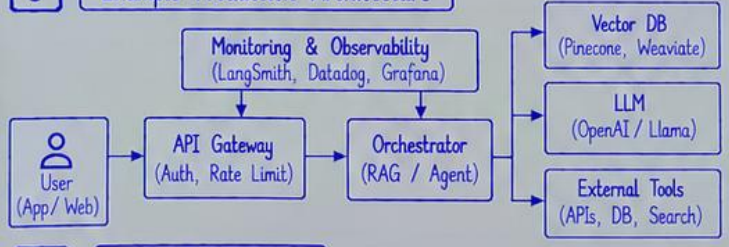
Tool	Use Case
LangSmith	Tracing, evaluation, monitoring for LLM apps
Weights & Biases	Experiment tracking, monitoring
OpenAI Evals	Evaluate LLM outputs
Arize Phoenix	LLM observability and monitoring
DeepEval	Evaluation framework
Helicone	Track token usage and API cost
Datadog	Infrastructure and application monitoring
Grafana	Visualize metrics and alerts
Prometheus	Metrics collection and alerting

codedsoul_05

8 Cost Optimization Techniques

- Use smaller and right-sized models for the task
- Implement caching (prompt cache, response cache)
- Use RAG to reduce context length instead of sending large prompts
- Chunk and filter data to send only relevant information
- Use prompt compression and token efficient prompting
- Batch requests when possible
- Monitor token usage and set budget alerts
- Use open source models for non critical tasks
- Optimize retrieval and reduce unnecessary tool calls

9 Example Production Architecture



10 Production Best Practices

- Start with a clear use case and success metrics
- Use evaluation and monitoring from day one
- Implement guardrails and safety checks
- Design for scalability and fault tolerance
- Keep human in the loop for critical use cases
- Set up cost monitoring and alerts
- Version prompts, models and data
- Log and trace all requests and responses
- Continuously improve using user feedback

11 Key Takeaways

- LLMOps helps build reliable, scalable and cost efficient LLM applications.
- Monitoring, evaluation and guardrails are essential for production.
- Use the right tools and architecture for your use case.
- Use agents collection and alerting.
- Control costs with efficient model usage, caching and token optimization.
- Production is an ongoing process with continuous monitoring and improvement.

LLM Handbook

20. LLM Cheat Sheet + 100 Interview Questions

1 Quick LLM Cheat Sheet

LLM	Large Language Model for understanding and generating text
Tokens	Smallest unit of text (word, subword or character)
Context Window	Maximum tokens model can process at once
Parameters	Internal weights learned during training
Temperature	Controls randomness (0 = deterministic, 1 = creative)
Top P (Nucleus)	Selects tokens from most probable cumulative probability
Max Tokens	Limits the length of output
System Prompt	Defines model behavior and role
User Prompt	Actual input from user
Few Shot	Provide examples in prompt
Zero Shot	Directly ask without examples
Chain of Thought	Ask model to think step by step
RAG	Retrieve external knowledge to improve answer
Fine Tuning	Train model on domain specific data
Embeddings	Convert text to vectors
Vector DB	Store and search embeddings (Pinecone, Weaviate, etc.)
Function Calling	Allow model to call external tools and APIs
Agents	Plan, use tools, take actions to achieve goals
Guardrails	Safety filters, moderation, validation
Evaluation	Measure quality, relevance and safety of output

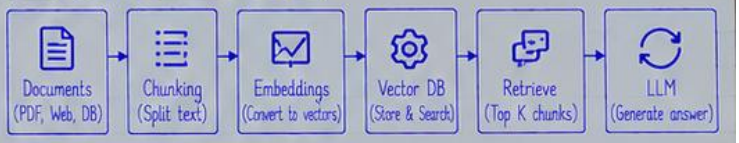
2 Popular LLM Models Comparison

Model	Provider	Context Window	Strengths	Use Case
GPT-4o	OpenAI	128K	General purpose	Chat, coding, vision
Claude 3.5	Anthropic	200K	Long context, safety	Analysis, long docs
Gemini 1.5	Google	1M	Multimodal, long context	Docs, images, video
Llama 3	Meta	8K/70B+	Open source, customizable	Research, fine tuning
Mistral	Mistral AI	32K	Fast, efficient	Production apps
Phi 3	Microsoft	128K	Small, lightweight	On device, edge
DeepSeek	DeepSeek	128K	Good reasoning, open source	Coding, research

3 Prompting Techniques

Zero Shot	Few Shot	Chain of Thought	Role Prompting	Structured Output
Direct question No examples	Give few examples	Ask to think step by step	Assign a role (e.g. expert)	Ask for JSON or specific format

4 RAG Cheat Sheet



5 Important Libraries and Tools

Category	Libraries / Tools
LLM & APIs	OpenAI, Anthropic, Google Gemini, TogetherAI, Groq
Frameworks	LangChain, LangGraph, LlamaIndex, AutoGen, CrewAI
Vector DB	Pinecone, Weaviate, Chroma, Qdrant, Milvus
Evaluation	Ragas, TruLens, DeepEval, OpenAI Evals, Helicone
Monitoring	LangSmith, Arize Phoenix, Datadog, Grafana, Prometheus
Deployment	Vercel, AWS, Docker, Kubernetes, Modal, RunPod
Other Tools	HuggingFace, MLflow, Weights & Biases, BentoML

6 Common Use Cases



7 Model Evaluation Metrics

Metric	What it Measures	When to Use
Accuracy	Correctness of answer	QA, classification
Relevance	How relevant the answer is	RAG, open ended QA
Faithfulness	Grounded in given context	RAG, factual answers
Helpfulness	Useful and complete response	Chatbots, assistants
Toxicity / Safety	Harmful or biased content	All applications
Latency	Time to generate response	Production systems
Cost	Token usage and infra cost	Production and scaling
Human Evaluation	Real user feedback	Critical and complex use cases

8 Production Checklist

- ☐ Define clear use case and success metrics
- ☐ Choose right model and architecture
- ☐ Implement RAG or fine tuning if needed
- ☐ Add guardrails and safety filters
- ☐ Set up monitoring and logging
- ☐ Track cost and token usage
- ☐ Load testing and scaling plan
- ☐ Evaluate with real user data
- ☐ Implement feedback loop
- ☐ Handle failures and fallbacks
- ☐ Ensure data privacy and compliance
- ☐ Continuously improve

9 Common Interview Topics (100 Questions Covered)

1. LLM Basics and Fundamentals
2. Tokenization and Embeddings
3. Prompting and In Context Learning
4. RAG and Information Retrieval
5. Fine Tuning, SFT, PEFT and LoRA
6. RLHF, DPO and Alignment
7. Function Calling and Tool Use
8. AI Agents and Agentic Workflows
9. LLM Evaluation and LLM as a Judge
10. Hallucination, Guardrails and Safety
11. LLMOps, Monitoring, Cost and Production
12. Vector Databases and Retrieval Systems
13. Scaling and Optimization Techniques
14. System Design for LLM Applications
15. Real World Use Cases and Case Studies
16. Framework and Tooling Questions
17. Model Comparison and Tradeoffs
18. Security, Privacy and Compliance
19. Research and Latest Trends
20. Behavioral and Scenario Based Questions

10 Sample Interview Questions (First 10)

codedsoul_05

1. What is an LLM and how does it work?
2. Explain tokenization and different tokenization algorithms.
3. What are embeddings and how are they used?
4. How does RAG work and what are its advantages?
5. When would you choose fine tuning over prompt engineering?
6. What is the difference between SFT, PEFT and LoRA?
7. Explain RLHF and DPO in simple terms.
8. How does function calling work in LLMs?
9. What are AI agents and how are they different from simple LLM applications?
10. How do you evaluate an LLM output and what metrics would you use?